

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ  
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

**Факультет информатики, математики и компьютерных наук**

**Программа подготовки магистров по направлению**

**01.04.02 Прикладная математика и информатика**

*Трутнев Алексей Игоревич*

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА**

Кластеризация в пространствах большой размерности

Рецензент

---

Научный руководитель  
д-р физико-математических  
наук, проф.

---

В.А. Калягин

Нижний Новгород, 2025

# Содержание

|          |                                                                                       |           |
|----------|---------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Введение</b>                                                                       | <b>4</b>  |
| <b>2</b> | <b>Обзор литературы</b>                                                               | <b>7</b>  |
| 2.1      | Введение в кластеризацию . . . . .                                                    | 7         |
| 2.2      | Методы кластеризации . . . . .                                                        | 8         |
| 2.2.1    | Кластеризация на основе центроидов . . . . .                                          | 9         |
| 2.2.2    | Кластеризация на основе плотности . . . . .                                           | 11        |
| 2.2.3    | Иерархическая кластеризация . . . . .                                                 | 12        |
| 2.2.4    | Кластеризация на основе обучения Хебба . . . . .                                      | 15        |
| 2.3      | Проблемы пространства большой размерности . . . . .                                   | 16        |
| 2.3.1    | Использование расстояния Минковского . . . . .                                        | 17        |
| 2.3.2    | Кластеризация с частичным привлечением учителя . . .                                  | 18        |
| 2.3.3    | Самоконтролируемая кластеризация . . . . .                                            | 19        |
| 2.3.4    | Методы снижения размерности . . . . .                                                 | 20        |
| <b>3</b> | <b>Постановка проблемы</b>                                                            | <b>30</b> |
| 3.1      | Основные обозначения . . . . .                                                        | 30        |
| 3.2      | Формулирование задачи кластеризации без учителя . . . . .                             | 30        |
| 3.2.1    | Выбор метода кластеризации . . . . .                                                  | 31        |
| 3.2.2    | Выбор метрики качества . . . . .                                                      | 32        |
| <b>4</b> | <b>Предложенный метод</b>                                                             | <b>35</b> |
| 4.1      | «Мягкое» привлечение учителя . . . . .                                                | 35        |
| 4.2      | «Жесткое» привлечение учителя . . . . .                                               | 36        |
| 4.3      | Алгоритм $k$ -Means с метрикой Минковского и частичным привлечением учителя . . . . . | 37        |
| <b>5</b> | <b>Заключение</b>                                                                     | <b>39</b> |



# 1 Введение

Современный мир характеризуется стремительным ростом объемов данных, генерируемых в самых разных областях человеческой деятельности: от научных исследований и финансовых транзакций до пользовательской активности в цифровом пространстве. В условиях столь интенсивного накопления информации особое значение приобретает задача её систематизации и извлечения скрытых закономерностей. Одним из наиболее эффективных методов в этом направлении является кластеризация — процесс группировки объектов в такие кластеры, внутри которых элементы схожи между собой и существенно отличаются от элементов других групп.

Кластеризация относится к числу методов обучения без учителя в машинном обучении и применяется в случаях, когда отсутствуют заранее размеченные данные или ясные категории. В отличие от классификации, где алгоритму заранее известны метки классов, кластеризация стремится выявить структуру данных без предварительных предположений о числе или характере групп. Это делает метод особенно актуальным для задач, требующих начального анализа или сегментации больших массивов информации.

Методы кластеризации находят широкое применение в самых различных отраслях. Например, в биоинформатике алгоритмы кластеризации используются для анализа экспрессии генов и выявления групп с одинаковыми биологическими характеристиками. В маркетинге и аналитике потребительского поведения кластеризация позволяет сегментировать клиентов по предпочтениям, активности или географическому положению, способствуя более точной настройке рекламных кампаний. В области информационного поиска и анализа текстов кластеризация применяется для группировки документов по тематике, выявления спам-сообщений и упрощения навигации по большим корпусам текста. Не менее важна кластеризация и в изучении социальных сетей, где она помогает обнаруживать сообщества пользователей и анализировать паттерны взаимодей-

ствий.

С технической точки зрения кластеризация представляет собой математическую задачу, включающую выбор метрики расстояния, определение числа кластеров (в случае методов вроде k-средних), а также интерпретацию полученных групп. Существуют различные подходы к кластеризации: от иерархических моделей до методов, основанных на плотности (DBSCAN) и распределениях (ЕМ-алгоритм). Каждый из них обладает своими преимуществами и ограничениями, что делает актуальным вопрос выбора алгоритма в зависимости от конкретных характеристик данных.

Однако, несмотря на универсальность и популярность, методы кластеризации сталкиваются с рядом фундаментальных ограничений. Одним из наиболее значимых является ухудшение качества кластеризации при увеличении размерности данных. Эта проблема особенно актуальна в пространствах высокой размерности, когда число признаков существенно превышает количество объектов. Такое явление получило название «проклятие размерности» (curse of dimensionality).

Проклятие размерности выражается в том, что по мере увеличения числа признаков расстояния между объектами становятся всё менее информативными: большинство точек оказываются на одинаковом удалении друг от друга, и различия между ними нивелируются. Это приводит к разреженности данных, и традиционные метрики сходства (например, евклидово расстояние), используемые в методах вроде k-средних, теряют свою эффективность. Более того, высокая размерность усложняет интерпретацию результатов, увеличивает вероятность переобучения и делает вычислительные алгоритмы менее устойчивыми к шуму и выбросам.

Одним из эффективных подходов к решению этой проблемы является предварительное снижение размерности. Методы такие как PCA (Principal Component Analysis), MDS (Multi Dimensional Scaling) и UMAP (Uniform Manifold Approximation and Projection) позволяют сократить пространство признаков при сохранении

его ключевой структуры. Однако выбор метода снижения размерности и его влияние на итоговое качество кластеризации остаются открытыми вопросами и требуют детального анализа.

В данной работе рассматривается применение кластеризации с предварительным применением методов снижения размерности и анализ их влияния на качество кластеризации, устойчивости алгоритма и интерпретируемости результатов.

Для достижения данной цели поставлены следующие задачи:

1. систематизировать и проанализировать существующие методы кластеризации, а также методы снижения размерности,
2. реализовать алгоритм кластеризации предварительным применением методов снижения размерности,
3. провести экспериментальное исследование и анализ влияния методов снижения размерности на качество кластеризации,
4. разработать рекомендации по выбору методов снижения размерности в зависимости от характеристик данных и целей кластеризации.

Таким образом, работа направлена на разработку и экспериментальное обоснование стратегии, сочетающей в себе преимущества методов снижения размерности и алгоритма  $k$ -средних. Такой подход позволяет не только минимизировать негативные эффекты высокой размерности, но и повысить точность и надёжность кластеризации в условиях больших и сложных наборов данных.

## 2 Обзор литературы

### 2.1 Введение в кластеризацию

Кластеризация представляет собой один из фундаментальных методов машинного обучения, используемый для группировки объектов на основе их внутренних признаков и сходства без привлечения учителя, то есть без использования размеченных данных. В отличие от классификации, где объекты распределяются по заранее определённым классам, кластеризация позволяет выявлять скрытые структуры и закономерности в данных, что делает её особенно полезной в ситуациях, когда заранее неизвестно, как должны выглядеть классы или группы.

Кластеризация применяется в широком спектре научных и прикладных задач, охватывая такие области, как анализ данных [36], компьютерное зрение [12], биоинформатика [23], обработка естественного языка [14], а также информационные технологии, социальные науки, медицина и финансовая аналитика. Её основная цель — разделить множество объектов на группы (кластеры) таким образом, чтобы объекты в пределах одного кластера обладали максимальной степенью сходства, в то время как объекты различных кластеров — были существенно различны.

Кластеризация может быть использована для решения различных задач, таких как:

- **Сегментация изображений:** разделение изображения на области схожих пикселей, что позволяет выделить объекты на изображении и улучшить качество последующего анализа.
- **Классификация текстов:** группировка текстовых документов по темам или стилям, что позволяет упростить поиск и анализ информации.
- **Кластеризация пользователей:** сегментация пользователей на группы

схожих интересов и предпочтений, что позволяет персонализировать рекомендации и маркетинговые стратегии.

- **Обнаружение аномалий:** выявление аномальных точек, не принадлежащих ни одному кластеру или попадающих в редкие группы, что может свидетельствовать о мошенничестве, сбоях в оборудовании или ошибках в данных.
- **Разведочный анализ данных (EDA, Exploratory Data Analysis):** выявление скрытых закономерностей и структур в данных, что особенно полезно при работе с новыми или плохо изученными наборами данных.

## 2.2 Методы кластеризации

Существуют различные подходы к реализации кластеризации, отличающиеся не только алгоритмической природой, но и способностью справляться с различными типами данных и структурами кластеров. В зависимости от метода формирования кластеров можно выделить следующие основные категории:

- **Центроидные методы** — основаны на минимизации расстояний между объектами и центральными точками кластеров (k-средних, k-медиан).
- **Иерархические методы** — формируют древовидную структуру кластеров путём последовательного объединения или разбиения групп (агломерационная и дивизионная кластеризация).
- **Плотностные методы** — строят кластеры как области повышенной плотности объектов (например, DBSCAN, OPTICS).
- **Модельные методы** — предполагают, что данные генерируются из смеси вероятностных распределений (например, ЕМ-алгоритм для гауссовых смесей).

- **Спектральные и графовые методы** — используют спектральные свойства графов и матриц смежности для обнаружения кластеров в данных со сложной топологией.

Каждый из подходов обладает своими достоинствами и ограничениями, что требует осознанного выбора метода в зависимости от конкретных характеристик исследуемых данных: размерности, плотности, формы кластеров и наличия шумов. Далее будут подробно рассмотрены эти методы, включая анализ их применимости, вычислительной сложности и устойчивости к внешним факторам.

### 2.2.1 Кластеризация на основе центроидов

Кластеризация на основе центроидов (centroid-based clustering) представляет собой один из наиболее интуитивно понятных и широко используемых подходов к группировке данных. Суть данного метода заключается в том, что каждый кластер описывается так называемым центроидом — обобщённым представлением «центра тяжести» данных в пределах соответствующего кластера. Объекты присваиваются к тому или иному кластеру на основе минимизации расстояния до соответствующего центроида.

Наиболее известным и базовым представителем центроидной кластеризации является алгоритм  $k$ -Means, впервые подробно описанный в работе Ллойда [26]. Данный алгоритм итеративно обновляет позиции центроидов и перераспределяет объекты между кластерами с целью минимизации внутрикластерной дисперсии — суммарного квадратичного расстояния между объектами и центрами кластеров. Простота реализации и высокая вычислительная эффективность обеспечили  $k$ -Means широкое распространение, особенно при работе с большими объемами данных.

Однако классический  $k$ -Means обладает рядом существенных ограничений. Во-первых, он чувствителен к начальной инициализации центроидов, что может приводить к сходимости к локальным минимумам. Во-вторых, алгоритм предпо-

лагает заданное заранее число кластеров  $k$ , что не всегда возможно определить априори. Кроме того, алгоритм плохо справляется с кластерами, имеющими сложную форму, различную плотность или отличающиеся размерами.

В ответ на указанные ограничения было предложено несколько модификаций. Один из таких подходов —  $k$ -Means++ [4], который улучшает этап начальной инициализации центроидов путём применения стохастической стратегии выбора начальных точек. Это повышает стабильность и качество итогового разбиения, особенно при наличии шумов и неоднородности в данных.

Альтернативой, сохраняющей идею центроидного представления, является алгоритм  $k$ -Medoids, также известный как PAM (Partitioning Around Medoids) [22]. В отличие от  $k$ -Means, где центроиды могут быть абстрактными точками,  $k$ -Medoids использует в качестве центров реальные объекты из выборки — медианы. Это повышает устойчивость к выбросам и позволяет использовать произвольные метрики расстояния, включая манхэттенскую, косинусную и др. Однако вычислительная сложность алгоритма существенно выше, особенно при больших объемах данных.

Среди более современных подходов можно выделить CLARANS (Clustering Large Applications based upon RANdomized Search) [32], который комбинирует элементы  $k$ -Medoids с методами стохастического поиска и улучшает масштабируемость алгоритма при работе с крупными выборками. В свою очередь, Mini-Batch  $k$ -Means [35] адаптирован для обработки потоков данных и масштабных задач за счёт обновления центроидов на основе случайных подвыборок (батчей), что уменьшает требования к памяти и ускоряет обучение.

Несмотря на ограниченность в обнаружении кластеров сложной формы, методы, основанные на центроидах, остаются актуальными благодаря своей простоте, интерпретируемости и высокой вычислительной эффективности. Они особенно полезны в качестве базовых моделей или в составе более сложных гибридных решений, где предварительная кластеризация используется как этап инициализации или предобработки данных.

### 2.2.2 Кластеризация на основе плотности

Методы кластеризации на основе плотности (density-based clustering) представляют собой эффективный класс алгоритмов, разработанных для выявления кластеров в данных с неоднородной плотностью распределения. Основная идея заключается в том, что кластеры формируются как связные области пространства с высокой плотностью объектов, отделённые друг от друга зонами разреженности. В отличие от методов, основанных на центроидах или иерархической структуре, плотностные алгоритмы способны выделять кластеры произвольной формы, устойчивы к шуму и выбросам, а также не требуют предварительного задания числа кластеров.

Наиболее известным и фундаментальным методом в этой категории является DBSCAN (Density-Based Spatial Clustering of Applications with Noise), предложенный в работе М. Эстера и соавторов [13]. Алгоритм определяет кластеры как максимальные области, содержащие взаимосвязанные точки с плотностью выше заданного порога. Основные параметры алгоритма – радиус окрестности и минимальное число точек в окрестности — определяют, является ли точка ядровой, граничной или шумовой. DBSCAN эффективно справляется с задачами обнаружения кластеров различных форм, но его производительность существенно зависит от корректного выбора параметров, что особенно проблематично при переменной плотности данных.

Для преодоления ограничений DBSCAN в условиях разной плотности кластеров был предложен алгоритм OPTICS (Ordering Points To Identify the Clustering Structure) [2]. В отличие от DBSCAN, OPTICS не производит окончательное разбиение, а строит упорядоченное представление плотности данных, называемое reachability plot. Это позволяет визуально определить плотностные структуры и формировать кластеры постфактум, адаптируясь к локальным особенностям плотности. OPTICS расширяет применимость методов плотностной кластеризации, особенно в случаях с вложенными и слабо выраженными структурами.

Более современным и мощным развитием идей DBSCAN является HDBSCAN (Hierarchical DBSCAN) [28]. Данный алгоритм строит иерархическую дендрограмму на основе плотностной связи между точками, а затем применяет процедуру извлечения стабильных кластеров. Он автоматически определяет оптимальное количество кластеров и значительно лучше справляется с кластерами переменной плотности, при этом наследуя устойчивость DBSCAN к шуму. HDBSCAN также предлагает оценку надежности принадлежности точки к кластеру, что повышает интерпретируемость результатов.

Другим интересным направлением является DENCLUE (DENsity-based CLUstering) [27], использующий ядерные оценки плотности (kernel density estimation) для моделирования распределения данных. Это позволяет формировать кластеры на основе градиентных потоков в плотностном пространстве и обеспечивает гибкую настройку формы и гладкости кластеров. Однако метод требует вычислительно дорогих операций и чувствителен к выбору параметра сглаживания.

Плотностные алгоритмы находят широкое применение в задачах обнаружения аномалий, анализе геопространственных данных, выявлении сообществ в социальных сетях и анализе временных рядов, где традиционные методы центроидного типа или иерархические подходы оказываются неэффективными.

В целом, подходы, основанные на плотности, являются мощными средствами анализа сложных, неструктурированных и шумных данных. Их гибкость и устойчивость делают их незаменимыми в случаях, когда структура кластеров заранее неизвестна и может отличаться высокой степенью нелинейности.

### **2.2.3 Иерархическая кластеризация**

Иерархическая кластеризация представляет собой мощный класс алгоритмов, ориентированных на поэтапное построение структуры вложенных кластеров без необходимости заранее задавать их количество. В отличие от центроидных и плотностных методов, иерархический подход формирует древовидную

структуру, называемую дендрограммой, где каждый уровень отражает определённый этап объединения или разбиения кластеров. Это делает метод особенно полезным для визуального анализа структуры данных и выбора оптимального количества кластеров после завершения обучения.

Существует два основных направления иерархической кластеризации:

- Агломеративная (снизу вверх) стратегия, при которой каждый объект изначально рассматривается как отдельный кластер, и далее происходит последовательное объединение ближайших кластеров вплоть до образования единой группы. Данный подход является наиболее распространённым и практически применимым, особенно в случае умеренных размеров выборки. Одним из самых популярных алгоритмов этой категории является метод Уорда [31], минимизирующий увеличение внутрикластерной дисперсии при каждом объединении.
- Дивизионная (сверху вниз) стратегия, напротив, начинается с единственного кластера, включающего все объекты, и рекурсивно разделяет его на подмножества. Несмотря на высокую концептуальную привлекательность, методы этого типа, например алгоритм, предложенный Кауфманом и Руссо [21], требуют значительно больших вычислительных затрат и применяются реже.

Качество работы иерархической кластеризации в значительной степени зависит от выбора метрики расстояния между кластерами. Существует несколько стратегий, определяющих, как именно измеряется расстояние между группами данных:

**Single Linkage (Метод ближнего соседа)** Метод ближнего соседа [37] определяет расстояние между кластерами как минимальное расстояние между парой точек из разных кластеров. Он хорошо справляется с кластеризацией вытянутых или нестандартных форм, однако чувствителен к шуму и может приводить

к так называемому «эффекту цепочки» — последовательному объединению разреженных объектов.

**Complete Linkage (Метод дальнего соседа)** В методе дальнего соседа [11] расстояние между кластерами определяется как максимальное расстояние между точками двух кластеров. Этот подход обеспечивает более компактные и равномерные кластеры, но при этом склонен переоценивать расстояния и может неправомерно разделять вытянутые структуры.

**Average Linkage (Метод средней связи)** Средняя связь [45] использует среднее значение расстояний между всеми парами точек из двух кластеров. Данный метод обеспечивает баланс между крайними подходами и демонстрирует устойчивость к шуму, но иногда недостаточно точно отражает геометрию данных.

Кроме того, иерархические методы удобны тем, что позволяют динамически выбирать оптимальное число кластеров, обрезая дендрограмму на нужной высоте без необходимости многократного запуска алгоритма. Это особенно ценно в условиях, когда структура данных неизвестна заранее.

Несмотря на высокую интерпретируемость и визуальную наглядность, недостатками иерархических методов являются высокая вычислительная сложность ( $O(n^2)$  или выше для большинства реализаций) и невозможность коррекции ошибок: однажды принятые решения о слиянии или разбиении не подлежат пересмотру.

Тем не менее, иерархическая кластеризация остаётся важным инструментом в арсенале анализа данных и активно применяется в биоинформатике, текстовом анализе, социологии, психологии и других дисциплинах, где требуется исследование сложных иерархических структур в выборке.

#### 2.2.4 Кластеризация на основе обучения Хебба

Кластеризация на основе обучения Хебба (Hebbian Learning) представляет биологически вдохновленный метод обучения, основывающийся на фундаментальном принципе нейрофизиологии, сформулированном Дональдом Хеббом в 1949 году [17]. Согласно так называемому правилу Хебба, «нейроны, которые активируются совместно, укрепляют связь между собой» (neurons that fire together, wire together). Эта идея легла в основу адаптивного обучения в искусственных нейронных сетях, а также стала мощным инструментом для кластеризации данных на основе корреляционной близости объектов.

С точки зрения формализации, классическое хеббовское обучение предполагает обновление весов связей по правилу:  $\Delta w = \eta x u$ , где  $x$  и  $u$  — входной и выходной сигналы,  $w$  — вес связи между входом и выходом, а  $\eta$  — коэффициент обучения. При многократном предъявлении данных такие схемы обучения приводят к усилению связей между часто совместно активируемыми признаками, что способствует формированию естественных кластеров.

Преимуществом хеббовских методов кластеризации является их биологическая правдоподобность, адаптивность к данным, способность выявлять латентные структуры без строгих априорных предположений о количестве кластеров. Однако они также обладают определёнными ограничениями. Наиболее существенными из них являются чувствительность к параметру обучения, зависимость от порядка предъявления входов, а также необходимость калибровки модели для достижения стабильной сходимости.

Тем не менее, обучение Хебба продолжает рассматриваться как перспективная альтернатива классическим методам кластеризации, особенно в условиях высокой размерности и отсутствия чётких границ между группами данных.

## 2.3 Проблемы пространства большой размерности

Анализ и кластеризация данных в пространствах высокой размерности остаются актуальными задачами современной науки о данных. По мере увеличения числа признаков (измерений), используемых для описания объектов, традиционные методы анализа, включая алгоритмы кластеризации, начинают сталкиваться с фундаментальными затруднениями. Это явление широко известно как «проклятие размерности» (curse of dimensionality), термин, введённый Ричардом Беллманом в 1957 году в контексте динамического программирования [7].

**Вычислительная сложность** Одним из наиболее очевидных последствий высокой размерности является экспоненциальный рост вычислительной нагрузки. Многие алгоритмы кластеризации, в частности  $k$ -Means, требуют вычисления расстояний между каждым объектом и каждым центроидом на каждой итерации. При этом сложность возрастает линейно по числу измерений и квадратично по числу объектов. В пространствах с тысячами признаков это приводит к существенному увеличению времени работы алгоритмов и ресурсов памяти, ограничивая масштабируемость [43].

**Разреженность данных** По мере увеличения размерности пространства наблюдается феномен разреженности — большая часть гиперкуба, в котором располагаются данные, оказывается практически пустой. Объекты становятся статистически эквивалентно удалёнными друг от друга, и плотность данных резко падает. Как показано в [8], различия в расстояниях между ближайшими и дальними соседями стремятся к нулю по мере роста размерности, делая евклидову метрику всё менее полезной. Это приводит к затруднениям в определении границ между кластерами, особенно при использовании методов на основе расстояния и плотности (например, DBSCAN).

**Снижение эффективности метрик** С увеличением размерности традиционные метрики расстояния теряют дискриминационную силу. Евклидово расстояние, косинусная близость и манхэттенская метрика становятся всё менее способными различать объекты, что подрывает эффективность как центроидных, так и иерархических и плотностных методов кластеризации [1]. Это особенно критично для алгоритмов, основанных на ближайших соседях или на построении локальных связей.

### 2.3.1 Использование расстояния Минковского

Одной из возможных стратегий повышения качества кластеризации в пространствах большой размерности является адаптация метрики расстояния. В этом контексте особое внимание заслуживает расстояние Минковского, которое представляет собой обобщение различных метрических функций:

$$d(x, y) = \left( \sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}}. \quad (1)$$

Значения параметра  $p$  позволяют интерполировать между манхэттенской ( $p = 1$ ) и евклидовой ( $p = 2$ ) метриками, а при  $p \rightarrow \infty$  метрика приближается к максимуму по координатам. Исследования показали, что выбор оптимального значения параметра  $p$  может существенно повлиять на эффективность кластеризации в условиях высокой размерности [1]. Особенно перспективным направлением является использование взвешенного расстояния Минковского, где каждый признак имеет индивидуальный вес:

$$d(x, y) = \left( \sum_{i=1}^n w_i |x_i - y_i|^p \right)^{\frac{1}{p}}, \quad (2)$$

где  $w_i$  — вес признака  $i$ . Эти веса могут быть получены, например, с помощью анализа важности признаков, извлечённой из моделей машинного обучения, либо с использованием разложения главных компонент (РСА). Такой подход

позволяет учитывать разную информативность признаков, что особенно важно в условиях высокой размерности и коррелированных признаков. Практические результаты подтверждают, что взвешенные метрики существенно улучшают устойчивость алгоритмов, таких как  $k$ -Means, к шуму и нерелевантным признакам [10].

### 2.3.2 Кластеризация с частичным привлечением учителя

Другая важная стратегия повышения эффективности кластеризации — это частичное привлечение учителя. Semi-supervised clustering сочетает преимущества обучаемых без учителя методов с ограниченным числом меток классов, что позволяет направлять процесс кластеризации к более интерпретируемым и структурированным результатам. Такой подход используется, например, при наличии априорной информации о небольшом числе объектов, чья принадлежность к кластерам заранее известна. Этот подход позволяет добавить контроль в процесс кластеризации, что приводит к созданию различных кластеров с более четкой структурой, а также добавляет возможность использовать стандартные методы оценки качества результатов алгоритма, такие как точность (Accuracy), прецизионность (Precision) полнота (Recall). Среди алгоритмов кластеризации с частичным привлечением учителя можно выделить основные подходы использования ограниченного знания о классах исходных данных.

**Привлечение учителя для генерации начальных кластеров** Подход кластеризации с частичным привлечением учителя для генерации начального разбиения на кластеры (seeding) [5], [3] — это метод, в котором небольшой набор маркированных точек данных используют для формирования начальных кластеров, таким образом предоставляя алгоритму хорошую стартовую точку для оптимизационной задачи. В дальнейшем алгоритм кластеризации уже без учителя распространит начальные метки на оставшиеся немаркированные данные на основе мер схожести, обеспечивая сильное влияние начального приближения

на весь процесс обучения.

**Привлечение учителя для определения принадлежности к кластеру** Одним из способов внести знания о кластерах в процесс обучения также является привлечение учителя для определения принадлежности к кластеру. Метод Semi-supervised clustering under a «compact-cluster» assumption [18], который основан на допущении идеи компактности кластеров и минимизирует внутрикластерные расстояния, одновременно максимизируя межкластерные расстояния, обеспечивает четкость и компактность кластеров.

### 2.3.3 Самоконтролируемая кластеризация

Самоконтролируемая кластеризация (self-supervised clustering) представляет собой гибридную парадигму, сочетающую принципы самоконтролируемого обучения (self-supervised learning, SSL) с задачей кластеризации. В её основе лежит идея извлечения устойчивых и структурно значимых признаков из неразмеченных данных, которые в дальнейшем могут использоваться для формирования кластеров с высокой интерпретируемостью и разделимостью. SSL зарекомендовал себя как эффективный инструмент предварительного анализа, способный существенно улучшать производительность downstream-задач, включая кластеризацию [15].

SSL представляет собой класс методов, в которых обучающая информация извлекается из самих данных без использования явных меток. В результате обучения создаются эмбединги — векторы признаков, которые можно использовать в задачах сегментации, визуализации и кластеризации.

Таким образом, SSL становится этапом предварительного анализа данных. Одним из таких методов может быть этап снижения размерности, позволяющий компактно представить данные в пространстве с меньшей размерностью, сохранив при этом ключевую топологическую структуру.

### 2.3.4 Методы снижения размерности

**Метод главных компонент (Principal Component Analysis)** Метод главных компонент (Principal Component Analysis, PCA) — это один из наиболее известных и широко используемых алгоритмов снижения размерности, предложенный ещё в начале XX века и формализованный в современном виде в классической работе Джоллиффа [20]. PCA позволяет перевести данные из пространства высокой размерности в подпространство меньшей размерности, сохранив при этом максимум возможной дисперсии данных. Важной особенностью метода является его линейная природа, что делает его особенно эффективным для задач, в которых основная информация содержится в линейных комбинациях признаков.

Основная идея PCA заключается в нахождении нового ортонормированного базиса пространства признаков, в котором направления (так называемые главные компоненты) упорядочены по убыванию дисперсии проекций данных. Первые несколько компонентов захватывают наибольшую вариативность данных и используются для проекции исходных точек в пространство меньшей размерности. Это позволяет уменьшить объём хранимой информации, ускорить обучение моделей и устранить мультиколлинеарность. Алгоритм PCA включает следующие этапы:

- Стандартизация признаков (центрация и нормализация).
- Вычисление ковариационной матрицы.
- Вычисление собственных значений и собственных векторов ковариационной матрицы.
- Сортировка собственных векторов по убыванию соответствующих собственных значений.
- Формирование проекционной матрицы из первых  $k$  векторов.

- Проецирование исходных данных в новое пространство признаков.

Пример реализации описан в алгоритме 1.

Простота PCA, его высокая вычислительная эффективность и математическая интерпретируемость сделали метод фундаментальным инструментом в таких областях, как: — обработка изображений (например, сжатие изображений, распознавание лиц [40]); — биоинформатика (анализ экспрессии генов [33]); — финансовое моделирование (анализ корреляционных структур рынков); — кластеризация и визуализация многомерных данных [19].

Однако, несмотря на достоинства, линейность PCA является его ограничением. Метод не способен корректно обрабатывать данные, в которых основная структура отражается в нелинейных зависимостях между признаками.

---

#### **Algorithm 1** Метод главных компонент (PCA)

---

**Require:** Матрица данных  $X \in \mathbb{R}^{m \times n}$  с  $m$  образцами и  $n$  признаками

**Шаг 1:** Стандартизация данных:  $X_{\text{std}} = \frac{X - \mu}{\sigma}$ , где  $\mu$  — среднее значение, а  $\sigma$  — стандартное отклонение каждого признака.

**Шаг 2:** Вычисление ковариационной матрицы:  $\Sigma = \frac{1}{n} X_{\text{std}}^T X_{\text{std}}$ .

**Шаг 3:** Вычисление собственных значений и собственных векторов ковариационной матрицы:  $\Sigma v_i = \lambda_i v_i$ , где  $\lambda_i$  — собственные значения, а  $v_i$  — собственные векторы.

**Шаг 4:** Сортировка собственных значений и соответствующих им собственных векторов в порядке убывания  $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$

**Шаг 5:** Выбор первых  $k$  собственных векторов для формирования матрицы проекции  $W \in \mathbb{R}^{n \times k}$ :  $W = [v_1, v_2, \dots, v_k]$

**Шаг 6:** Преобразование исходных данных в новое подпространство:  $Z = X_{\text{std}} W$

**return** Матрица данных с уменьшенной размерностью  $Z \in \mathbb{R}^{m \times k}$  с  $k$  главными компонентами.

---

**Метод многомерного шкалирования (Multidimensional Scaling)** Метод многомерного шкалирования (Multidimensional Scaling, MDS) — это классический подход к снижению размерности, направленный на сохранение попарных расстояний или близостей между объектами при отображении их в пространство меньшей размерности. В отличие от метода главных компонент (PCA), MDS мо-

жет использовать любой тип метрик, включая неевклидовы, и поэтому считается более гибким для анализа данных с нелинейной структурой.

MDS был впервые разработан как способ визуализации психологической близости между объектами на основе субъективных оценок [25]. Позднее метод получил широкое распространение в информационных науках, биоинформатике, социальной аналитике, анализе текстов и других дисциплинах, где структура данных может быть выражена через матрицу попарных расстояний. Алгоритм MDS включает несколько этапов (2):

- Входные данные представляют собой матрицу попарных расстояний  $D$  между объектами.
- Сначала вычисляется матрица квадратов попарных расстояний  $B$  с помощью формулы  $B = -\frac{1}{2}D^2$ .
- Затем матрица  $B$  центрируется, чтобы устранить влияние глобального смещения и обеспечить корректное представление расстояний.
- Далее вычисляются собственные значения и собственные векторы центрированной матрицы  $B$ .
- Собственные значения сортируются в порядке убывания, и отбираются первые  $k$  собственных векторов, соответствующие наибольшим собственным значениям.
- Наконец, исходные данные проецируются в новое пространство размерности  $k$  с помощью матрицы проекции  $W$ , которая формируется из первых  $k$  собственных векторов.

Результатом является линейное преобразование, аналогичное PCA, но построенное на расстояниях, а не ковариациях.

---

**Algorithm 2** Метод многомерного шкалирования (MDS)

---

**Require:** Матрица попарных расстояний  $D \in \mathbb{R}^{m \times m}$  между  $m$  объектами

**Шаг 1:** Вычисление матрицы квадратов попарных расстояний:  $B = -\frac{1}{2}D^2$

**Шаг 2:** Центрирование матрицы  $B$ :  $B = B - \frac{1}{m}\mathbf{1}B - \frac{1}{m}B\mathbf{1} + \frac{1}{m^2}\mathbf{1}B\mathbf{1}$ , где  $\mathbf{1}$  – матрица, заполненная единицами.

**Шаг 3:** Вычисление собственных значений и собственных векторов матрицы  $B$ :  $Bv_i = \lambda_i v_i$ , где  $\lambda_i$  – собственные значения, а  $v_i$  – собственные векторы.

**Шаг 4:** Сортировка собственных значений и соответствующих им собственных векторов в порядке убывания  $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_m$ .

**Шаг 5:** Выбор первых  $k$  собственных векторов для формирования матрицы проекции  $W \in \mathbb{R}^{m \times k}$ :  $W = [v_1, v_2, \dots, v_k]$ .

**Шаг 6:** Преобразование исходных данных в новое подпространство:  $Z = W\Lambda^{1/2}$ , где  $\Lambda$  – диагональная матрица, содержащая  $k$  наибольших собственных значений.

**return** Матрица данных с уменьшенной размерностью  $Z \in \mathbb{R}^{m \times k}$  с  $k$  измерениями.

---

**Метод Uniform Manifold Approximation and Projection (UMAP)** Метод Uniform Manifold Approximation and Projection (UMAP) [30], представляет собой современный и эффективный нелинейный алгоритм снижения размерности, ориентированный на сохранение локальной и глобальной структуры данных. В отличие от классических линейных методов (например, PCA), UMAP обладает высокой масштабируемостью, вычислительной эффективностью и устойчивостью к шуму, что делает его востребованным в задачах визуализации, кластеризации и предварительной обработки данных. Ключевые шаги алгоритма UMAP:

- Построение  $k$ -графа ближайших соседей для каждого объекта и определение вероятностного распределения локальных расстояний.
- Построение матрицы локальных связей с использованием параметров расстояния и плотности.
- Инициализация точек в пространстве меньшей размерности (обычно случайным образом или с использованием метода PCA).

- Проведение оптимизации с помощью стохастического градиентного спуска, минимизирующей перекрытие между графами высокого и низкого размерностей.

UMAP позволяет сохранять как локальные, так и глобальные структуры данных. Алгоритм представлен ниже в форме псевдокода (алгоритм 3).

---

**Algorithm 3** Метод Uniform Manifold Approximation and Projection (UMAP)

---

**Require:** Матрица данных  $X \in \mathbb{R}^{m \times n}$  с  $m$  образцами и  $n$  признаками

**Шаг 1:** Вычисление матрицы ближайших соседей и вероятностной структуры локального графа

**Шаг 2:** Построение глобального графа связи (средства соединения локальных графов)

**Шаг 3:** Инициализация точек в проекционном пространстве

**Шаг 4:** Оптимизация размещения точек с использованием перекрытия распределений

**return** Матрица данных с уменьшенной размерностью  $Z \in \mathbb{R}^{m \times k}$  с  $k$  измерениями.

---

**Метод Isometric Mapping (Isomap)** Метод Isometric Mapping (Isomap) — это нелинейный алгоритм снижения размерности [39]. Isomap расширяет классический метод многомерного шкалирования (MDS), позволяя сохранить геодезические расстояния между точками на многообразии, а не только их евклидову дистанцию. Это делает метод особенно полезным для обработки данных, лежащих на нелинейных многообразиях высокой размерности.

Ключевая идея Isomap состоит в том, чтобы заменить евклидовы расстояния между объектами на приближённые геодезические расстояния, которые более точно отражают топологическую структуру данных на искривлённом многообразии. Геодезические расстояния определяются как кратчайшие пути вдоль поверхности (графа), аппроксимирующей пространство объектов. После этого используется метод классического MDS для получения координат в пространстве пониженной размерности. Алгоритм Isomap включает следующие шаги:

- Построение графа ближайших соседей, где для каждого объекта выбираются  $k$  ближайших соседей.

- Вычисление геодезических расстояний: определяется кратчайшее расстояние между всеми парами объектов по построенному графу с использованием, например, алгоритма Флойда–Уоршелла или Дейкстры.
- Применение MDS: проводится классическое многомерное шкалирование на полученной матрице геодезических расстояний для проекции данных в пространство меньшей размерности.

Isomap позволяет эффективно обрабатывать данные с нелинейной структурой, сохраняя при этом важные топологические свойства. Однако метод требует значительных вычислительных ресурсов и может быть чувствителен к шуму в данных. Алгоритм Isomap представлен ниже в форме псевдокода (алгоритм 4).

---

#### **Algorithm 4** Метод Isometric Mapping (Isomap)

---

**Require:** Матрица данных  $X \in \mathbb{R}^{m \times n}$  с  $m$  образцами и  $n$  признаками

**Шаг 1:** Построение графа ближайших соседей (по  $k$  ближайшим)

**Шаг 2:** Вычисление геодезических расстояний между всеми парами точек (алгоритм Флойда/Дейкстры)

**Шаг 3:** Применение MDS к матрице геодезических расстояний

**return** Матрица данных с уменьшенной размерностью  $Z \in \mathbb{R}^{m \times k}$  с  $k$  измерениями.

---

**Метод Locally Linear Embedding (LLE)** Метод Locally Linear Embedding (LLE) — это эффективный алгоритм нелинейного снижения размерности [34]. Он позволяет получить компактное представление высокоразмерных данных, сохранив при этом локальную геометрию исходного пространства. В отличие от линейных методов, таких как PCA, LLE способен выявлять нелинейные структуры, что делает его особенно полезным для обработки данных, лежащих на искривлённых многообразиях.

Идея LLE заключается в следующем: каждый объект в высокоразмерном пространстве аппроксимируется линейной комбинацией своих ближайших соседей. Предполагается, что на достаточно малых масштабах многообразие можно считать почти линейным. Эти веса затем используются для восстановления

представления объекта уже в пространстве меньшей размерности, с сохранением локальной топологии. Метод LLE включает следующие этапы:

- Поиск ближайших соседей для каждого объекта. Обычно используется метрика евклидова расстояния и выбирается фиксированное число соседей  $k$ .
- Вычисление весов локальной линейной аппроксимации. Для каждого объекта находятся коэффициенты  $w_{ij}$ , минимизирующие ошибку реконструкции  $x_i \approx \sum_j w_{ij} x_j$ , где  $x_j$  — ближайшие соседи точки  $x_i$ . Условия:  $\sum_j w_{ij} = 1$ ,  $w_{ij} = 0$ , если  $x_j$  не является соседом  $x_i$ .
- Обратная реконструкция: минимизация расстояния между исходными весами и их аналогами в пространстве меньшей размерности — нахождение точек  $y_i$ , минимизирующих  $\sum_i \|y_i - \sum_j w_{ij} y_j\|^2$ .
- Решение задачи через собственные значения: нахождение низкоразмерного представления  $Y$ , соответствующего наименьшим собственным значениям результирующей матрицы.

---

**Algorithm 5** Метод Locally Linear Embedding (LLE)

---

**Require:** Матрица данных  $X \in \mathbb{R}^{m \times n}$  с  $m$  образцами и  $n$  признаками

**Шаг 1:** Найти  $k$  ближайших соседей для каждого  $x_i$

**Шаг 2:** Решить задачу линейной аппроксимации и найти веса  $w_{ij}$

**Шаг 3:** Построить матрицу весов  $W$  (разреженную)

**Шаг 4:** Выбор случайных точек в пространстве меньшей размерности

**Шаг 5:** Найти проекции  $y_i$ , минимизируя функцию ошибки реконструкции  $\sum_i \|y_i - \sum_j w_{ij} y_j\|^2$

**return** Матрица данных с уменьшенной размерностью  $Y \in \mathbb{R}^{m \times k}$  с  $k$  измерениями.

---

**Метод Spectral Embedding (SSE)** Метод Spectral Embedding (SSE), также известный как спектральное разложение графа или Laplacian Eigenmaps, представляет собой мощный инструмент нелинейного снижения размерности [6]. SSE опирается на представление данных в виде графа, в котором вершины

соответствуют объектам, а рёбра — их близости. Метод позволяет сохранить локальную геометрию данных, что делает его особенно эффективным для задач, связанных с визуализацией, сегментацией и кластеризацией структурно сложных данных.

Основной идеей метода является построение взвешенного графа ближайших соседей, после чего рассчитывается его нормализованный лапласиан, отражающий топологическую структуру данных. Далее проводится спектральное разложение лапласиана, и проекции объектов на собственные векторы, соответствующие наименьшим собственным значениям, формируют новое пространство признаков пониженной размерности. Метод SSE включает следующие этапы:

- Построение матрицы сходства  $S$ : используется гауссово ядро или функция ближайших соседей, чтобы определить степень связи между объектами.
- Формирование матрицы Лапласиана  $L$ :  $L = D - S$ , где  $D$  — диагональная матрица степеней вершин,  $D_{ii} = \sum_j S_{ij}$ .
- Вычисление собственных значений и собственных векторов матрицы  $L$ : выбираются  $k$  собственных векторов, соответствующих наименьшим ненулевым собственным значениям.
- Формирование проекционного пространства: используется матрица  $W \in \mathbb{R}^{m \times k}$ , содержащая выбранные собственные векторы.
- Получение проекции:  $Z = W\Lambda^{1/2}$ , где  $\Lambda$  — диагональная матрица собственных значений.

---

**Algorithm 6** Метод Spectral Embedding (SSE)

---

**Require:** Матрица данных  $X \in \mathbb{R}^{m \times n}$  с  $m$  образцами и  $n$  признаками

**Шаг 1:** Построение матрицы сходства  $S$

**Шаг 2:** Формирование матрицы Лапласиана  $L = D - S$

**Шаг 3:** Спектральное разложение  $L : Lv_i = \lambda_i v_i$

**Шаг 4:** Формирование матрицы проекции  $W = [v_1, \dots, v_k]$

**Шаг 5:** Получение низкоразмерного представления  $Z = W\Lambda^{1/2}$

**return** Матрица данных с уменьшенной размерностью  $Z \in \mathbb{R}^{m \times k}$  с  $k$  измерениями.

---

## Обобщение

В ходе проведённого обзора были рассмотрены основные алгоритмы кластеризации, применяемые в задачах анализа многомерных данных. Одной из ключевых проблем, выявленных при этом, стало существенное снижение качества кластеризации в условиях высокой размерности признакового пространства. Такая ситуация, характерная для многих современных наборов данных, приводит к явлению, известному как проклятие размерности, при котором традиционные алгоритмы, включая  $k$ -Means, демонстрируют нестабильность, низкую точность результатов. Для преодоления этих ограничений активно применяются различные стратегии предварительной обработки данных, ключевую роль среди которых играет снижение размерности. Данный подход позволяет не только сократить вычислительную сложность алгоритмов, но и повысить устойчивость и точность кластеризации за счёт устранения шумовых и коррелированных признаков. Существует широкий спектр методов снижения размерности, начиная от линейных алгоритмов, таких как метод главных компонент (PCA) и метод многомерного шкалирования (MDS), и заканчивая нелинейными подходами, включая UMAP, Isomap, Locally Linear Embedding (LLE) и Spectral Embedding (SSE). Линейные методы, как правило, обеспечивают высокую интерпретируемость и простоту реализации, однако ограничены в возможности работы с данными, лежащими на искривлённых многообразиях. Нелинейные же подходы позво-

ляют более точно передавать сложные топологические структуры, но требуют аккуратной настройки и значительных вычислительных ресурсов. Несмотря на разнообразие существующих методов и их широкое применение, на практике остаётся ряд открытых проблем. Во-первых, не существует универсального критерия выбора метода снижения размерности для конкретной задачи кластеризации. Выбор подхода часто осуществляется эмпирически и зависит от множества факторов: характера данных, плотности распределения, предполагаемого количества кластеров и других параметров. Во-вторых, влияние предварительного понижения размерности на итоговое качество кластеризации далеко не всегда очевидно и требует тщательного эмпирического анализа. В-третьих, интеграция методов снижения размерности в общую процедуру кластеризации до сих пор часто остаётся фрагментарной и недостаточно формализованной.

## 3 Постановка проблемы

### 3.1 Основные обозначения

В дальнейшем будут использоваться следующие основные обозначения. Пусть  $X = \begin{pmatrix} x_i^j \end{pmatrix} \in \mathbb{R}^{n \times d}$  обозначает матрицу наблюдений размером  $n \times d$ , где каждый  $d$ -мерный объект  $x_i = (x_i^1, x_i^2, \dots, x_i^d) \in \mathbb{R}^{1 \times d}$  представляет собой строковый вектор признаков для каждого наблюдения  $i = 1, 2, \dots, n$ . Вектор  $x_j = (x_1^j, x_2^j, \dots, x_n^j)^\top \in \mathbb{R}^{n \times 1}$  означает столбец значений признака  $j$  для всех наблюдений  $j = 1, 2, \dots, d$ . Вектор  $y = (y_1, y_2, \dots, y_n)^\top \in \mathbb{R}^{n \times 1}$  содержит значения целевой функции, представляющей искомые кластеры. Число кластеров обозначено как  $k$ , а разбиение наблюдений на кластеры — как  $S = S_1, S_2, \dots, S_k$ , где каждый кластер  $S_l \subset 1, 2, \dots, n$  для  $l = 1, 2, \dots, k$ . Набор из  $k$  центров кластеров представлен через  $c = c_1, c_2, \dots, c_k$ , где каждый центр  $c_l = (c_l^1, c_l^2, \dots, c_l^d) \in \mathbb{R}^{1 \times d}$  для  $l = 1, 2, \dots, k$ . Также используется диагональная матрица размерности  $d$ , обозначенная как  $I_d$ .

### 3.2 Формулирование задачи кластеризации без учителя

Задача кластеризации относится к типу задач «без учителя» (Unsupervised learning). Задача кластеризации – есть группировка объектов в наборы (группы), основываясь только на их сходстве между собой так, чтобы объекты внутри одной группы были больше похожи на представителей этой группы, чем другой.

Формальная запись задач имеет следующий вид: Для каждого  $d$ -мерного объекта  $x_i$  из множества объектов  $X = x_1, x_2, \dots, x_n$ , необходимо поставить в соответствие один из  $k$  кластеров  $c = \{c_1, c_2, \dots, c_k\}$ .

Для оценивания «схожести» объектов могут быть использованы разные метрики расстояния такие как Евклидово расстояние [26], Манхеттенское расстояние, [22], расстояние Минковского [1] и другие. А также различные способы оценки межкластерных расстояний, такие как Average Linkage [45], Complete

Linkage [11], Single Linkage [37].

Задача кластеризации на основе центроидов для расстояния Минковского с параметром  $p$  есть задача сложной невыпуклой оптимизации функционала качества (3), которая может быть решена с помощью эвристического алгоритма  $k$ -Means для метрики Минковского, где необходимо оптимизировать сумму расстояний между объектами внутри кластеров.

$$\begin{aligned} W_p(S; c) &= \sum_{l=1}^k \sum_{x \in S_l} \sum_{j=1}^d |x^j - c_l^j|^p \rightarrow \min, \\ x &= (x^1, x^2, \dots, x^d) \in R^{1 \times d}, \\ c_l &= (c_l^1, c_l^2, \dots, c_l^d) \in R^{1 \times d} \end{aligned} \tag{3}$$

### 3.2.1 Выбор метода кластеризации

В данной работе основой для алгоритма кластеризации с учителем служит реализация  $k$ -Means с метрикой Минковского для пространств большой размерности, поскольку  $k$ -Means – легко интерпретируемый подход, основанный на оптимизации суммарного квадратичного отклонения внутри кластеров. Выбор  $k$ -Means также обусловлен его основными преимуществами: простотой реализации (существует большое число реализаций и оптимизаций данного подхода), высокой скоростью работы (на небольших наборах данных алгоритм способен сходиться за очень короткое время). Алгоритм  $k$ -Means выполняет кластеризацию данных, оптимизируя суммарное квадратичное отклонение внутри кластеров. Его работа состоит из нескольких этапов:

1. Инициализация. Фиксируется параметр  $k$  – число кластеров, которые необходимо найти, после чего центроиды кластеров выбираются случайным образом.
2. Присвоение данных кластерам. Каждый объект данных присваивается кластеру, чей центроид на основе выбранной метрики расстояния ока-

жется ближе всего.

3. Обновление центроидов. Центроиды пересчитываются на основе объектов из соответствующих кластеров.
4. Итеративное повторение шагов 1, 2 и 3.

### 3.2.2 Выбор метрики качества

Для оценки качества предложенного алгоритма кластеризации были выбраны метрики *Accuracy*, *NMI*, *ARI*, *AMI*.

**Rand Index (*RI*)** является метрикой, используемой для измерения сходства между двумя различными кластеризациями или разбиениями одного и того же набора данных. Основная идея *RI* заключается в том, чтобы подсчитать долю пар объектов, которые находятся в одном и том же кластере в обоих разбиениях (кластеризациях). Для того чтобы определить *RI*, вводятся следующие обозначения. Пусть  $X$  – набор, состоящий из  $n$  объектов для кластеризации, в котором  $a$  пар объектов попали в один кластер, а  $b$  пар попали в разные кластеры. Тогда метрика Rand Index (*RI*) будет иметь вид:

$$RI = \frac{a + b}{C_2^n}. \quad (4)$$

**Adjusted Rand Index (*ARI*)** – скорректированная версия Rand Index, предназначенная для измерения сходства между двумя кластеризациями, путем корректировки метрики *RI*. *ARI* может принимать значения в промежутке  $[0; 1]$ , где значение 1 соответствует идентичным кластеризациям, а значение 0 означает, что кластеризации случайны.

$$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]}, \quad (5)$$

где  $E[RI]$  - оценка математического ожидания *RI*, а  $\max(RI)$  - максимальное значение Rand Index, полученное в результате кластеризаций.

**Mutual Information (MI)** измеряет степени зависимости между двумя различными разбиениями одного и того же набора данных на кластеры. Обозначим за  $U$  и  $V$  два разбиения набора данных  $X$ , состоящего из  $n$  объектов, что  $|U_i|$  – есть количество объектов в кластере  $U_i$ , а  $|V_j|$  в кластере  $V_j$ . Обозначим энтропию за  $H(U)$ .

$$MI(U, V) = \sum_{i=1}^{|U|} \sum_{j=1}^{|V|} \frac{|U_i \cap V_j|}{n} \log \frac{n|U_i \cap V_j|}{|U_i||V_j|}, \quad (6)$$

**Normalized Mutual Information (NMI)** – вариант  $MI$ , который нормализуется для того, чтобы значения можно было интерпретировать на отрезке от 0 до 1. Метрика используется для измерения сходства между двумя различными разбиениями одного и того же набора данных на кластеры.

$$NMI(U, V) = \frac{MI(U, V)}{mean(H(U), H(V))}. \quad (7)$$

**Adjusted Mutual Information (AMI)** нормализуется относительно максимальной величины, которую может принять взаимная информация, что позволяет оценивать сходство между разбиениями на независимых от размера кластеров данных. Она принимает значения в интервале от 0 до 1, где значение 1 означает идентичные разбиения, а значение 0 указывает на полное отсутствие сходства. Данная метрика широко используется в оценке качества различных методов кластеризации или сегментации данных, особенно когда требуется оценить сходство результатов, полученных из разных экспериментов или алгоритмов кластеризации.

$$AMI(U, V) = \frac{MI(U, V) - E[MI(U, V)]}{mean(H(U), H(V)) - E[MI(U, V)]}. \quad (8)$$

**Accuracy** используется для оценки качества классификации, рассматривая степень совпадения найденных и референтных значений. Для оценки качества кластеризации без учителя использование Accuracy может не давать корректных

результатов, так как она опирается только на номера меток, которые в кластеризации могут различаться из-за различной нумерации кластеров. Однако при использовании подхода кластеризации с частичным привлечением учителя, где порядок нумерации кластеров совпадает с референтными значениями, данная метрика отражает насколько точно разделяются объекты на кластеры (классы).

## 4 Предложенный метод

Одним из способов повышения точности алгоритма кластеризации на основе центроидов для метрики Минковского является частичное привлечение учителя. В данной работе предлагается два способа введения информации о метках в алгоритм кластеризации.

### 4.1 «Мягкое» привлечение учителя

Мягкое привлечение учителя в задачу кластеризации (Soft semi-supervised clustering) – подход, предлагающий использовать информацию о метках в обучаемом наборе данных для генерации начальных центров вместо выбора их случайным методом. Для расчета начальных центроидов используется алгоритм (7), где каждый центроид высчитывается на основе размеченных объектов. Данный подход позволяет задать начальное приближение задачи оптимизации (3), которая в дальнейшем решается алгоритмом  $k$ -Means для метрики Минковского.

---

**Algorithm 7** Кластеризация с предварительным снижением размерности

---

$k$  : Количество классов

$X$  : Множество объектов

$d_{target}$  : Размерность пространства, в котором будет производиться кластеризация

$C = \{C_1, C_2, \dots, C_k\}$  : Разбиение на кластеры

$c = \{c_1, c_2, \dots, c_k\}$  : Центры кластеров

$Processor()$  : Выбранный метод снижения размерности

$X_{d_{target}} \leftarrow Processor(X, d_{target})$  - применить метод снижения размерности в пространство размерности  $d_{target}$  для данных  $X$

Выбрать случайным образом  $k$  центров кластеров из  $X_{d_{target}}$

**while** центры не стабилизировались **do**

**for** точка  $x \in X$  **do**

        вычислить расстояние до каждого центра  $c$

        присвоить  $x$  кластеру с ближайшим центром кластера

**end for**

**for** кластер  $C_i \in C$  и центр  $c_i \in c$  **do**

        пересчитать центр  $c_i$  кластера как среднее значение из точек внутри кластера  $C_i$

**end for**

**end while**

---

## 4.2 «Жесткое» привлечение учителя

Жесткое привлечение учителя в задачу кластеризации (Hard semi-supervised clustering) – подход, предлагающий расширить использование информации о метках в обучаемом наборе данных. В данном подходе предлагается использовать начальное приближение из алгоритма (7), а также «фиксировать» объекты с известными метками в их кластерах, фактически запрещая переходы объектов из одного кластера в другой, что влияет как на точность, так и на скорость схождения алгоритма.

---

**Algorithm 8** Выбор оптимального кластера для объекта

---

$x$  : объект

$T = \{t_1, t_2, \dots, t_r\}$  : Множество меток для  $r$  объектов

$C = \{C_1, C_2, \dots, C_k\}$  : Разбиение на кластеры

$c = \{c_1, c_2, \dots, c_k\}$  : Центры кластеров

**if**  $T$  содержит метку  $t$  для объекта  $x$  **then**

    присвоить объект  $x$  кластеру  $C_t$

**else**

    вычислить  $d_l$  - расстояние до каждого центра  $c_l \in c$

    присвоить объект  $x$  к ближайшему кластеру из  $C$

**end if**

---

### **4.3    Алгоритм $k$ -Means с метрикой Минковского и частичным привлечением учителя**

Таким образом, алгоритм кластеризации с метрикой Минковского и частичным привлечением учителя имеет следующий вид.

---

**Algorithm 9**  $k$ -Means с метрикой Минковского и частичным привлечением учителя

---

$k$  : Количество классов

$X$  : Множество объектов

$T = \{t_1, t_2, \dots, t_r\}$  : Множество меток для  $r$  объектов

$C = \{C_1, C_2, \dots, C_k\}$  : Разбиение на кластеры

$c = \{c_1, c_2, \dots, c_k\}$  : Центры кластеров

Выбрать начальное разбиение с помощью алгоритма (7)

**while** центры не стабилизировались **do**

**for** точка  $x \in X$  **do**

        присвоить  $x$  оптимальному кластеру согласно алгоритму (8)

**end for**

**for** кластер  $C_i \in C$  и центр  $c_i \in c$  **do**

        пересчитать центр  $c_i$  кластера как среднее значение из точек внутри кластера  $C_i$

**end for**

**end while**

---

Алгоритм был реализован при использовании языка программирования Python 3.10 и библиотеки *numpy* (1.24.3), *scikit-learn* (1.2.0), *scipy* (1.10.0), *umap-learn* (0.5.7). Для анализа результатов была разработана система автоматической генерации таблиц и графиков с использованием библиотеки *pandas* (1.5.2) и *matplotlib* (3.6.2). Для анализа качества алгоритма была разработана система автоматического анализа кода и тестирования алгоритма на платформе GitHub Actions. Качество кода проверялось с использованием библиотек *pylint* (2.15.9) и *isort* (5.11.4). Для тестирования алгоритма были разработаны 168 Unit тестов с использованием библиотеки *pytest* (7.2.0).

## 5 Заключение

В работе предлагаются и сравниваются два подхода введения учителя в процесс кластеризации  $k$ -Means для метрики Минковского для пространств большой размерности *SoftSSLKMeans* и *HardSSLKMeans*. *SoftSSLKMeans* использует идею введения учителя для генерации начального разбиения размеченных объектов на кластеры. *HardSSLKMeans* фиксирует метки размеченных объектов и не дает им сменить кластер в процессе решения задачи оптимизации. Оба алгоритма показывают значительно лучшие результаты в сравнении с алгоритмом  $k$ -Means без учителя. В частности, на 3 кластерах *SoftSSLKMeans* с 20% размеченными данными в среднем превосходит  $k$ -Means на 74.18%, *HardSSLKMeans* с 20% размеченными данными в среднем превосходит  $k$ -Means на 53.56%. На 2 кластерах *HardSSLKMeans* с 15% размеченными данными в среднем превосходит  $k$ -Means на 57%. Исследование выполнено с использованием суперкомпьютерного комплекса НИУ ВШЭ [24]. Код проекта доступен по ссылке [github.com/alexgiving/LKMeans](https://github.com/alexgiving/LKMeans).

## Список литературы

- [1] C. C. Aggarwal, A. Hinneburg, and D. A. Keim, “On the surprising behavior of distance metrics in high dimensional spaces,” in *Proceedings of the 8th International Conference on Database Theory*, ser. ICDT '01. Berlin, Heidelberg: Springer-Verlag, 2001, p. 420–434.
- [2] M. Ankerst, M. M. Breunig, H.-P. Kriegel, and J. Sander, “Optics: Ordering points to identify the clustering structure,” *SIGMOD Rec.*, vol. 28, no. 2, p. 49–60, jun 1999. [Online]. Available: <https://doi.org/10.1145/304181.304187>
- [3] V. Antoine, N. Labroche, and V.-V. Vu, “Evidential seed-based semi-supervised clustering,” in *2014 Joint 7th International Conference on Soft Computing and Intelligent Systems (SCIS) and 15th International Symposium on Advanced Intelligent Systems (ISIS)*, 2014, pp. 706–711.
- [4] D. Arthur and S. Vassilvitskii, “k-means++: the advantages of careful seeding,” in *SODA '07: Proceedings of the eighteenth annual ACM-SIAM symposium on Discrete algorithms*. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics, 2007, pp. 1027–1035.
- [5] S. Basu, A. Banerjee, and R. J. Mooney, “Semi-supervised clustering by seeding,” in *Proceedings of the Nineteenth International Conference on Machine Learning*, ser. ICML '02. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2002, p. 27–34.
- [6] M. Belkin and P. Niyogi, “Laplacian eigenmaps and spectral techniques for embedding and clustering,” *Advances in Neural Information Processing System*, vol. 14, 04 2002.
- [7] R. Bellman, *Dynamic Programming*, 1st ed. Princeton, NJ, USA: Princeton University Press, 1957.

- [8] K. Beyer, J. Goldstein, R. Ramakrishnan, and U. Shaft, “When is “nearest neighbor” meaningful?” in *Database Theory — ICDT’99*, C. Beeri and P. Buneman, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 217–235.
- [9] R. Cordeiro de Amorim and B. Mirkin, “Minkowski metric, feature weighting and anomalous cluster initializing in k-means clustering,” *Pattern Recognition*, vol. 45, no. 3, pp. 1061–1075, 2012. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0031320311003517>
- [10] R. C. de Amorim and P. Komisarczuk, “On initializations for the minkowski weighted k-means,” in *Advances in Intelligent Data Analysis XI*, J. Hollmén, F. Klawonn, and A. Tucker, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 45–55.
- [11] D. Defays, “An efficient algorithm for a complete link method,” *The Computer Journal*, vol. 20, no. 4, pp. 364–366, 01 1977. [Online]. Available: <https://doi.org/10.1093/comjnl/20.4.364>
- [12] A. Demidovskij, A. Tugaryov, A. Trutnev, M. Kazyulina, I. Salnikov, and S. Pavlov, “Lightweight and elegant data reduction strategies for training acceleration of convolutional neural networks,” *Mathematics*, vol. 11, no. 14, 2023. [Online]. Available: <https://www.mdpi.com/2227-7390/11/14/3120>
- [13] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, ser. KDD’96. AAAI Press, 1996, p. 226–231.
- [14] M. Grootendorst, “Bertopic: Neural topic modeling with a class-based tf-idf procedure,” 03 2022.

- [15] J. Gui, T. Chen, J. Zhang, Q. Cao, Z. Sun, H. Luo, and D. Tao, “A survey on self-supervised learning: Algorithms, applications, and future trends,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 9052–9071, 2024.
- [16] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, Sept. 2020. [Online]. Available: <https://doi.org/10.1038/s41586-020-2649-2>
- [17] D. Hebb, “The organization of behavior: A neuropsychological theory,” 1949.
- [18] Z. Jiang, Y. Zhan, Q. Mao, and Y. Du, “Semi-supervised clustering under a “compact-cluster” assumption,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 5, pp. 5244–5256, 2023.
- [19] I. Jolliffe and J. Cadima, “Principal component analysis: A review and recent developments,” *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 374, p. 20150202, 04 2016.
- [20] I. Jolliffe, *Principal Component Analysis*. Springer Verlag, 1986.
- [21] L. Kaufman and P. Rousseeuw, *Finding Groups in Data: An Introduction To Cluster Analysis*, 01 1990.
- [22] L. Kaufman and P. J. Rousseeuw, *Partitioning Around Medoids (Program PAM)*. John Wiley & Sons, Inc., 2008, pp. 68–125. [Online]. Available: <http://dx.doi.org/10.1002/9780470316801.ch2>

- [23] V. Kiselev, T. Andrews, and M. Hemberg, “Challenges in unsupervised clustering of single-cell rna-seq data,” *Nature Reviews Genetics*, vol. 20, p. 1, 01 2019.
- [24] P. Kostenetskiy, R. Chulkevich, and V. Kozyrev, “Hpc resources of the higher school of economics,” *Journal of Physics: Conference Series*, vol. 1740, p. 012050, 01 2021.
- [25] J. B. Kruskal, “Nonmetric multidimensional scaling: A numerical method,” *Psychometrika*, vol. 29, pp. 115–129, 1964. [Online]. Available: <https://api.semanticscholar.org/CorpusID:11709679>
- [26] S. Lloyd, “Least squares quantization in pcm,” *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [27] Y. Luo, K. Zhang, Y. Chai, and Y. Xiong, “Multi-parameter-setting based on data original distribution for denclue optimization,” *IEEE Access*, vol. 6, pp. 16 704–16 711, 2018.
- [28] L. McInnes, J. Healy, and S. Astels, “hdbscan: Hierarchical density based clustering,” *The Journal of Open Source Software*, vol. 2, 03 2017.
- [29] ———, “hdbscan: Hierarchical density based clustering,” *The Journal of Open Source Software*, vol. 2, 03 2017.
- [30] L. McInnes, J. Healy, and J. Melville, “Umap: Uniform manifold approximation and projection for dimension reduction,” 2020. [Online]. Available: <https://arxiv.org/abs/1802.03426>
- [31] F. Murtagh and P. Legendre, “Ward’s hierarchical agglomerative clustering method: Which algorithms implement ward’s criterion?” *Journal of Classification*, vol. 31, no. 3, pp. 274–295, Oct 2014. [Online]. Available: <https://doi.org/10.1007/s00357-014-9161-z>

- [32] R. Ng and J. Han, “Clarans: A method for clustering objects for spatial data mining,” *Knowledge and Data Engineering, IEEE Transactions on*, vol. 14, pp. 1003–1016, 10 2002.
- [33] M. Ringnér, “What is principal component analysis?” *Nature Biotechnology*, vol. 26, pp. 303–4, 04 2008.
- [34] S. Roweis and L. Saul, “Nonlinear dimensionality reduction by locally linear embedding,” *Science (New York, N.Y.)*, vol. 290, pp. 2323–6, 01 2001.
- [35] D. Sculley, “Web-scale k-means clustering,” in *Proceedings of the 19th International Conference on World Wide Web*, ser. WWW ’10. New York, NY, USA: Association for Computing Machinery, 2010, p. 1177–1178. [Online]. Available: <https://doi.org/10.1145/1772690.1772862>
- [36] S. Setyaningsih, “Using cluster analysis study to examine the successful performance entrepreneur in indonesia,” *Procedia Economics and Finance*, vol. 4, pp. 286–298, 2012, international Conference on Small and Medium Enterprises Development with a Theme ?Innovation and Sustainability in SME Development? (ICSMED 2012). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2212567112003437>
- [37] R. Sibson, “SLINK: An optimally efficient algorithm for the single-link cluster method,” *The Computer Journal*, vol. 16, no. 1, pp. 30–34, 01 1973. [Online]. Available: <https://doi.org/10.1093/comjnl/16.1.30>
- [38] B. Sorscher, R. Geirhos, S. Shekhar, S. Ganguli, and A. Morcos, “Beyond neural scaling laws: beating power law scaling via data pruning,” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 19 523–19 536.

- [39] J. B. Tenenbaum, V. de Silva, and J. C. Langford, “A global geometric framework for nonlinear dimensionality reduction,” *Science*, vol. 290, no. 5500, pp. 2319–2323, 2000. [Online]. Available: <https://www.science.org/doi/abs/10.1126/science.290.5500.2319>
- [40] M. Turk and A. Pentland, “Eigenfaces for recognition,” *Journal of Cognitive Neuroscience*, vol. 3, no. 1, pp. 71–86, 1991.
- [41] L. van der Maaten and G. Hinton, “Visualizing data using t-sne,” *Journal of Machine Learning Research*, vol. 9, no. 86, pp. 2579–2605, 2008. [Online]. Available: <http://jmlr.org/papers/v9/vandermaaten08a.html>
- [42] G. Van Rossum and F. L. Drake, *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009.
- [43] M. Verleysen and D. François, “The curse of dimensionality in data mining and time series prediction,” in *International Work-Conference on Artificial and Natural Neural Networks*, 2005. [Online]. Available: <https://api.semanticscholar.org/CorpusID:9343744>
- [44] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, Í. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [45] N. Xu, R. B. Finkelman, S. Dai, C. Xu, and M. Peng, “Average linkage hierarchical clustering algorithm for determining the relationships between

elements in coal,” *ACS Omega*, vol. 6, no. 9, pp. 6206–6217, 2021, pMID: 33718711. [Online]. Available: <https://doi.org/10.1021/acsomega.0c05758>